Marion Charpier



Introduction au langage de programmation Python

Séance 3 - Programmation orientée objet 1 : créer ses propres objets

Programme du cours

Séance	Date	Sujet
1	29 septembre 2026	Les bases de la programmation
2	6 octobre 2026	Créer ses propres outils : fonctions et modules
3	14 octobre 2026	Programmation orientée objet 1 : créer ses propres objets
4	20 octobre 2026	Programmation orientée objet 2 : héritage et décorateurs
5	27 octobre 2026	Lire et enregistrer des données : JSON
6	3 novembre 2026	Du tableur à Python : CSV
7	10 novembre 2026	Explorer les collections numérisées : IIIF

Plan de la séance 3

1) **Correction de l'interrogation**
2) **Les modules en Python**
   a) Qu'est-ce qu'un module ?
   b) Importation de modules et conventions
   c) Quelques modules utiles
   d) Créer son propre module
   e) Organiser ses modules : les packages
3) **Introduction aux classes et aux objets**
   a) Qu'est-ce que la POO ?
   b) Les classes
   c) Les objets
4) **Les attributs et les méthodes**
   a) La méthode __init__, le paramètre self et les attributs d'instance
   b) Les attributs de classe
   c) Attributs de classe ou d'instance ?



Correction de l'interrogation (1/2)

Q.	Réponse	Pourquoi
1	titre.split()	Une méthode s'écrit après la donnée, avec un point. len(), print() et sorted() sont des fonctions : la donnée est entre les parenthèses.
2	XIIe siècle puis None	La fonction affiche avec print(), mais ne renvoie rien : valeur contient donc None.
3	latin	L'appel ne fournit pas d'argument pour langue : la valeur par défaut "latin" s'applique.
4	creer_notice(cote="MS 24", 1200)	Un argument positionnel ne peut pas suivre un argument nommé : SyntaxError: positional argument follows keyword argument.
5	3	"MS 24" va dans le paramètre cote ; *noms regroupe les trois créatures restantes. Piège : 4 compte aussi la cote.

Correction de l'interrogation (2/2)

Q.	Réponse	Pourquoi
6	dict	**infos regroupe les arguments nommés dans un dictionnaire. C'est *args qui donne un tuple.
7	Python renvoie une erreur	resultat est locale à siecle() et disparaît après l'appel : NameError: name 'resultat' is not defined.
8	III, 14. De la colombe.	L'affectation dans la fonction crée une nouvelle variable locale : la variable globale titre n'est pas modifiée.
9	La signature puis la docstring	help() n'exécute pas la fonction : on lui donne la fonction elle-même, sans parenthèses ni argument.
10	# Un siècle va de l'an 1 à l'an 100 : 1200 est au XIIe siècle	Il explique **pourquoi** on a écrit <=. Les autres paraphrasent le code ou n'apportent aucune information.

Les modules en Python



Définition

Un **module** est un fichier .py qui contient des fonctions, des variables ou des classes regroupées autour d'un même thème. Il permet de **réutiliser**, d'**organiser** et de **partager** du code. Python fournit de nombreux modules prêts à l'emploi.

Importer un module entier : import

```
import random
random.choice(creatures)   # 'licorne' (tirage au hasard)
```

On accède au contenu du module par un point : random.choice(). Le point va chercher un élément à l'intérieur de ce qui le précède : dans un module, comme ici, ou dans une donnée, comme titre.split().

Avantage : on voit d'où vient chaque fonction, et on évite les conflits de noms.

Importer un élément précis : from … import …

```
from math import ceil
ceil(1210 / 100)   # 13, le numéro du siècle
```

L'élément s'utilise directement, sans préfixe.
Attention : un nom importé peut être écrasé par mégarde.

```
from random import choice

choice = "licorne"
choice(creatures)
# TypeError: 'str' object is not callable
```

Donner un alias : import … as …

```
import pandas as pd

pd.DataFrame(corpus)
```

L'alias raccourcit les noms longs. Certains sont des conventions partagées par tous.

Où ? Les importations se placent **en tête** du fichier ou du notebook.

Module, package, bibliothèque : importer proprement



Comprendre la différence

- **Module** : un **fichier** .py, comme un chapitre consacré à un sujet précis. Exemples : random, csv.
- **Package** : un **dossier** qui regroupe plusieurs modules, comme un volume réunissant plusieurs chapitres (slide 12).
- **Bibliothèque** : un **ensemble** de modules et de packages distribué d'un seul tenant, comme un livre entier.
    - La **bibliothèque standard** est livrée avec Python : random, math, pathlib, datetime… On n'a rien à installer.
    - Les **bibliothèques externes** s'installent avec pip : pandas, matplotlib, requests…

Bonnes pratiques d'importation

Respecter les alias conventionnels

pd pour pandas, np pour numpy, plt pour matplotlib.pyplot. Tout le monde les reconnaît.

Grouper les importations, dans cet ordre

```
# Bibliothèque standard
import random
from pathlib import Path

# Bibliothèques externes
import pandas as pd
import matplotlib.pyplot as plt

# Modules locaux (les nôtres)
import bestiaire
```

```
import tensorflow as plt
import pandas as tf
import numpy as pd
import matplotlib.pyplot as np
```

Quelques modules de la bibliothèque standard



1. pathlib : fichiers et dossiers

Manipuler des chemins, lister le contenu d'un dossier, trouver des fichiers.

```
from pathlib import Path

dossier = Path("images")        # dossier d'images numérisées
```

Chercher des fichiers avec glob()

glob() est une méthode de Path : elle renvoie tous les fichiers d'un dossier dont le nom correspond à un **motif**.

- * remplace n'importe quelle suite de caractères ;
- ? remplace un seul caractère.

```
for image in sorted(dossier.glob("*.jpg")):
    print(image.name)
# MS24_f001r.jpg
# MS24_f001v.jpg
```

glob() ne renvoie pas une liste, mais une suite de fichiers que l'on parcourt avec une boucle for ou que l'on entoure de sorted() pour obtenir les fichiers dans l'ordre alphabétique.

2. random : le hasard

Tirer un élément, un nombre, mélanger une liste.

```
import random

random.choice(creatures)     # une créature au hasard
random.randint(1, 10)        # un entier entre 1 et 10 inclus
random.shuffle(creatures)    # mélange la liste elle-même
```

À venir

Deux autres modules de la bibliothèque standard structureront la suite du cours :

- json (séance 5) pour lire et enregistrer des données au format JSON ;
- csv (séance 6) pour passer du tableur à Python.

Pour aller plus loin : la documentation de la bibliothèque standard, docs.python.org/fr/3/library

Quelques bibliothèques externes



Installer une bibliothèque externe

Les bibliothèques externes ne sont pas livrées avec Python : il faut les installer une fois avec pip.

```
pip install pandas          # dans le terminal
%pip install pandas         # dans une cellule de notebook
```

Sans installation, l'importation échoue :
`ModuleNotFoundError: No module named 'pandas'`

1. requests : interroger le web

Télécharger une page ou des données depuis une adresse web, par exemple une API.

```
import requests

reponse = requests.get("https://www.example.com")
print(reponse.status_code)   # 200 si la requête a réussi
```

On s'en servira en séance 7 pour interroger les collections numérisées avec IIIF.

2. pandas : les tableaux de données

Manipuler des données en lignes et en colonnes, comme dans un tableur.

```
import pandas as pd

tableau = pd.DataFrame(corpus)
print(tableau)
#            cote                bibliotheque  date langue  numerise
# 0          MS 24  Aberdeen, University Library  1200  latin
True
# 1  MS. Ashmole 1511      Oxford, Bodleian Library  1210
NaN      True
```

Chaque notice devient une ligne et chaque clé une colonne. `NaN` signale une valeur manquante : la notice d'Oxford n'a pas de clé "langue".

On s'en servira en séance 6 pour passer du tableur à Python.

Créer son propre module

Un module personnalisé

C'est un fichier .py dans lequel on range ses propres fonctions et données, pour les réutiliser dans plusieurs notebooks ou scripts sans les recopier.

Les étapes

1. **Créer un fichier** bestiaire.py dans VS Code, **dans le même dossier** que le notebook.
2. **Y écrire** une docstring de module, puis ses fonctions et ses données.
3. **Enregistrer** le fichier. Il est alors importable avec import bestiaire.

Nommer son module

Le nom de fichier sert de nom au module : il doit être en minuscules, sans accents ni espaces, et ne pas reprendre le nom d'un module existant (random.py, math.py…).

Une donnée fixe : une constante

CREATURES est écrit en majuscules : par convention, c'est une **constante**, une valeur qu'on ne modifie pas.

```
# bestiaire.py
"""Fonctions et données pour le corpus de bestiaires médiévaux."""

CREATURES = ["dragon", "licorne", "lion", "panthère", "phénix"]

def siecle(date):
    """Renvoyer le siècle d'une date sous forme de texte."""
    if date <= 1100:
        …
    return resultat

def cote_complete(manuscrit):
    """Renvoyer la bibliothèque et la cote d'un manuscrit."""
    return f"{manuscrit['bibliotheque']}, {manuscrit['cote']}"
```

Importer et utiliser son module

Importer le module entier

```
import bestiaire

for ms in corpus:
    print(bestiaire.cote_complete(ms), ":",
bestiaire.siecle(ms["date"]))
# Aberdeen, University Library, MS 24 : XIIe siècle
# Oxford, Bodleian Library, MS. Ashmole 1511 : XIIIe siècle
```

Importer des éléments précis

```
from bestiaire import siecle, CREATURES

siecle(1150)      # 'XIIe siècle'
len(CREATURES)    # 5
```

Notre module s'importe exactement comme random ou pathlib (slide 8).

Où Python cherche-t-il bestiaire.py ?

D'abord dans le **dossier du notebook** ou du script, puis dans la bibliothèque standard et les bibliothèques installées. Si le fichier est ailleurs, on obtient :

```
ModuleNotFoundError: No module named 'bestiaire'
```

Attention : modifier un module déjà importé

Python n'importe un module **qu'une fois**. Si l'on modifie bestiaire.py ensuite, refaire import bestiaire ne change rien. Deux solutions :

- **redémarrer le noyau** du notebook (bouton *Restart* dans VS Code), puis relancer les cellules ;
- ou recharger le module :

```
import importlib
importlib.reload(bestiaire)
```

Organiser ses modules : les packages

Module ou package ?

- Un **module** est un seul fichier .py.
- Un **package** est un **dossier** qui regroupe plusieurs modules et un fichier __init__.py.

Pourquoi des packages ?

- **Structurer** : quand un projet grandit, plusieurs petits fichiers rangés par thème sont plus faciles à gérer qu'un seul gros fichier.
- **Isoler** : chaque module s'occupe d'une partie du travail, ce qui limite les interférences.
- **Partager** : un package se transmet et se réutilise d'un projet à l'autre. C'est ainsi que sont organisées les bibliothèques comme pandas.

Exemple : si notre module bestiaire.py grandissait

```
bestiaires/
├── __init__.py
├── dates.py       # siecle()
└── notices.py     # creer_notice(), cote_complete()
```

Importer depuis un package : le point sépare le package du module.

```
from bestiaires.dates import siecle
siecle(1210)              # 'XIIIe siècle'

import bestiaires.notices
bestiaires.notices.cote_complete(manuscrit)
# 'Aberdeen, University Library, MS 24'
```

Le fichier `__init__.py`

Qu'est-ce que c'est ?

`__init__.py` est un fichier placé dans un dossier pour indiquer à Python que ce dossier est un **package**. Python l'exécute au moment où l'on importe le package.

Est-il obligatoire ?

Non, depuis Python 3.3 : un dossier sans `__init__.py` s'importe quand même. On le met pourtant **par convention**, car il signale clairement « ceci est un package ».

Le cas le plus courant : un fichier vide

Un `__init__.py` vide suffit. C'est ce qu'on fera dans la plupart des projets.

Usage possible : simplifier les importations

```
# bestiaires/__init__.py
"""Outils pour le corpus de bestiaires médiévaux."""

from .dates import siecle
from .notices import cote_complete
```

Le point dans `.dates` signifie « le module dates **de ce même package** ». On peut alors écrire :

```
from bestiaires import siecle
```

au lieu de :

```
from bestiaires.dates import siecle
```

Bonne pratique : garder `__init__.py` court et simple. Un code lourd placé ici ralentit chaque importation du package.

La Programmation Orientée Objet



Définition

La programmation orientée objet (POO) est une façon d'organiser son code autour d'**objets**. Un objet réunit des **données**, appelées **attributs**, et des **actions**, appelées **méthodes**, qui portent sur ces données.

Pourquoi la POO ?

- **Modéliser** : un objet Manuscrit ressemble à ce qu'il représente.
- **Organiser** : chaque classe forme une unité que l'on développe et teste séparément. Cela aide à gérer la complexité et à travailler à plusieurs.
- **Réutiliser** : une classe sert d'un programme à l'autre, sans recopier de code.

Maintenir : une modification de la classe vaut pour tous ses objets.

Attributs et méthodes

- **Attributs** : ce que l'objet possède, par exemple sa cote, sa bibliothèque ou sa date. Ce sont les **noms**.
- **Méthodes** : ce que l'objet sait faire, par exemple renvoyer sa cote complète. Ce sont les **verbes**.

De la notice à l'objet

Jusqu'ici : un dictionnaire, et des fonctions à part.

```
manuscrit["cote"]          # 'MS 24'
cote_complete(manuscrit)   # 'Aberdeen, University Library, MS 24'
```

Avec la POO : données et actions réunies dans l'objet.

```
manuscrit.cote             # 'MS 24'
manuscrit.cote_complete()  # 'Aberdeen, University Library, MS 24'
```

Le point fonctionne comme dans `titre.split()` : une chaîne de caractères est déjà un objet, avec ses méthodes.

Les classes

Qu'est-ce qu'une classe ?

Une **classe** est un **modèle** pour créer des objets. Elle définit les attributs (données) et les méthodes (actions) que posséderont tous les objets créés à partir d'elle. Autrement dit, elle décrit ce qu'est un objet et ce qu'il peut faire.

La classe Manuscrit décrit ce qu'ont en commun tous les manuscrits du corpus : une cote, une bibliothèque, une date.

Syntaxe

```
class Manuscrit:
    pass
```

- class : le mot-clé qui définit une classe.
- Manuscrit : le nom de la classe.
- : : les deux-points, qui ouvrent le bloc de la classe.
- **Le corps de la classe**, indenté, contient ses attributs et ses méthodes.

Conventions de nommage : PascalCase

Le nom d'une classe suit la convention **PascalCase** : chaque mot commence par une majuscule, sans séparateur.

Manuscrit, NoticeCatalogue, CreatureFabuleuse

Les variables et les fonctions, elles, s'écrivent en minuscules, avec des tirets bas : manuscrit, cote_complete.

Pourquoi cette convention ?

- **Lisibilité** : on distingue d'un coup d'œil une classe d'une variable ou d'une fonction. Manuscrit est le modèle, manuscrit est un manuscrit précis. Python respecte la casse : ce sont deux noms différents.
- **Conformité** : c'est la convention de toute la communauté Python. La respecter facilite la collaboration et la lecture du code des autres.

Les objets

Qu'est-ce qu'un objet ?

Un **objet** est une réalisation concrète d'une classe. On dit aussi une **instance** : l'objet est une instance de la classe. Créer un objet, c'est **instancier** la classe.

Classe et objet

- La **classe** est le modèle : elle décrit ce que tout manuscrit possède et sait faire.
- L'**objet** est un manuscrit précis, avec ses propres valeurs : MS 24 à Aberdeen, MS. Ashmole 1511 à Oxford. Chaque objet existe indépendamment des autres.

Colonne de droite

Instancier une classe

1. **Définir** la classe.
2. **Créer** l'objet : on appelle la classe avec des parenthèses, comme une fonction.
3. **Utiliser** l'objet : accéder à ses attributs, appeler ses méthodes.

```
class Manuscrit:
    pass

manuscrit = Manuscrit()
print(manuscrit)

# <__main__.Manuscrit object at 0x7f3a2c1d5e10>
```

Python affiche la classe de l'objet et son adresse en mémoire, qui change à chaque exécution. L'objet existe, mais il est encore vide : c'est `__init__` qui lui donnera des attributs (slide 17).

Les attributs de classe

```
class Manuscrit:
    def __init__(self, cote, bibliotheque, date):
        self.cote = cote
        self.bibliotheque = bibliotheque
        self.date = date

    def cote_complete(self):
        return f"{self.bibliotheque}, {self.cote}"

corpus = [
    Manuscrit("MS 24", "Aberdeen, University Library", 1200),
    Manuscrit("MS. Ashmole 1511", "Oxford, Bodleian Library", 1210),
]
for manuscrit in corpus:
    print(manuscrit.cote_complete())
# Aberdeen, University Library, MS 24
# Oxford, Bodleian Library, MS. Ashmole 1511
```

Les attributs d'instance

self.cote, self.bibliotheque et self.date sont **propres à chaque objet**. On y accède par objet.attribut, et modifier ceux d'un objet n'affecte pas les autres.

La méthode __init__

Appelée **automatiquement** à la création de l'objet, elle **initialise** ses attributs. On la dit souvent « constructeur », mais elle ne crée pas l'objet : elle le remplit.

Le paramètre self

- Il désigne **l'objet lui-même**. Python le passe automatiquement : on ne l'écrit pas à l'appel.
- self.cote = cote range la valeur dans l'objet. Sans self., elle serait perdue à la fin de la méthode.
- self est une convention de nom, à toujours respecter.

Une méthode

C'est une fonction définie dans la classe, avec self comme premier paramètre. D'où manuscrit.cote_complete(), sans argument.

Les attributs d'instance

Qu'est-ce qu'un attribut de classe ?

C'est une variable définie **directement dans la classe**, en dehors des méthodes. Elle est **partagée par toutes les instances**. On l'utilise pour une valeur commune à tous les objets.

```
class Manuscrit:
    genre = "bestiaire"

    def __init__(self, cote, bibliotheque, date):
        self.cote = cote
        self.bibliotheque = bibliotheque
        self.date = date
```

Accès : par la classe ou par une instance.

```
Manuscrit.genre     # 'bestiaire'
corpus[0].genre     # 'bestiaire'
corpus[1].genre     # 'bestiaire'
```

Modifier par la classe : tous les objets changent

```
Manuscrit.genre = "bestiaire latin"
corpus[0].genre     # 'bestiaire latin'
corpus[1].genre     # 'bestiaire latin'
```

Modifier par une instance : un seul objet change

```
corpus[0].genre = "bestiaire enluminé"
corpus[0].genre     # 'bestiaire enluminé'
corpus[1].genre     # 'bestiaire latin'
```

Attention : l'affectation par une instance ne modifie pas l'attribut de classe. Elle crée un **attribut d'instance** du même nom, propre à cet objet, qui masque l'attribut de classe.

Différence entre attributs de classe et attributs d'instance

	Attributs de classe	Attributs d'instance
Définition	Définis dans la classe, en dehors des méthodes	Définis dans __init__, avec self
Partage	Partagés par toutes les instances	Propres à chaque instance
Accès	Classe.attribut ou instance.attribut	instance.attribut
Modification	Par la classe : affecte toutes les instances	N'affecte que l'instance concernée
Utilisation	Valeurs communes à tous les objets	Valeurs propres à chaque objet
Exemple	genre = "bestiaire"	self.cote, self.date

Notions clés à retenir : les modules



- **Module** : un fichier .py.
  - **Package** : un dossier de modules avec __init__.py.
  - **Bibliothèque** : un ensemble distribué.
- **Importer** : import module, from module import element, import module as alias, en tête de fichier.
- **Bibliothèque standard**, déjà installée : pathlib, random, math, et json (séance 5) et csv (séance 6).
- **Bibliothèques externes**, à installer avec pip : requests, pandas.
- **Son propre module** : bestiaire.py, placé dans le dossier du notebook et importé avec import bestiaire. Après une modification, il faut redémarrer le noyau ou utiliser importlib.reload().

Notions clés à retenir : la POO

Les classes

- Des **modèles** pour créer des objets.
- Un nom en **PascalCase** : Manuscrit.

Les objets

- Des **instances** de classes : manuscrit = Manuscrit("MS 24", ...).
- Ils possèdent des **attributs** (les noms) et des **méthodes** (les verbes).

Les attributs

- **De classe** : définis directement dans la classe, **partagés par toutes les instances**.
- **D'instance** : initialisés dans __init__ avec self, **propres à chaque objet**.

Les méthodes

- Des fonctions définies dans la classe, avec self en premier paramètre : manuscrit.cote_complete().

La méthode __init__

- Elle est appelée **automatiquement** à l'instanciation, et initialise les attributs d'instance.

Le paramètre self

- Il désigne **l'objet lui-même**. Python le passe automatiquement.